

red team your LLM.

THE OPEN SOURCE LLM SCANNER FROM NVIDIA

150+

ATTACK PROBES

24+

VULNERABILITY CATEGORIES

12

LLM GENERATOR BACKENDS

5min

TO FIRST FINDING

WHAT'S INSIDE

01 – 03

FOUNDATION

what · architecture · install

04 – 13

10 PROBE FAMILIES

the attacks that matter most

14 – 16

OPERATIONS

generators · filtering · reports

10 PROBES YOU'LL LEARN

- encoding
- dan
- leakreplay
- malwaregen
- goodside
- promptinject
- divergence
- packagehallucination
- xss
- ansiescape

NVIDIA AI Red Team · Apache 2.0 · github.com/NVIDIA/garak

garak

WHAT IS • WHY IT EXISTS

01 / 16

```
root@localhost:~# garak --version
garak v0.10.0 (NVIDIA AI Red Team)
Apache 2.0 · github.com/NVIDIA/garak
```

- ▶ what: LLM vulnerability scanner
- akin to: nmap, metasploit – for LLMs
- targets: any LLM via API or local

```
✓ tests for:
    prompt injection · jailbreaks
    data leakage · malware gen
    encoding bypass · supply chain
    ANSI escape · invisible inject
```

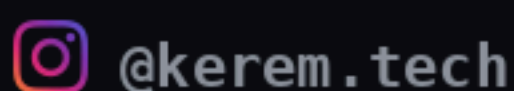
PROFILE

```
built by      NVIDIA AI Red Team  ·  Apache 2.0
inspired by   nmap      ·  Metasploit Framework
scope         150+ probes   ·  3000+ adversarial prompts
```

OVERVIEW

garak probes language models for known failure modes: prompt injection, jailbreaks, data leakage, toxicity, malware generation, encoding bypass. It brings nmap-style systematic security testing to AI systems via static, dynamic, and adaptive probes.

Real impact: used by Databricks, NVIDIA, and AI security teams worldwide.
Recommended as industry standard in independent reviews.



architecture

02 / 16

HOW GARAK WORKS

```
garak - scan - zsh

root@localhost:~# garak --list_config

generators      12      LLM backends to test
probes          150+    attack modules
detectors       40+    output classifiers
bufs            6      input mutations
harnesses       2      scan orchestrators

► scan flow:
  generator → probe → detector → eval
    target      attack      judgment
```

COMPONENTS

generators	backend to call the target LLM
probes	attack and test cases per category
detectors	classify responses, mark hits
bufs	input mutations (translate, encode, paraphrase)

OVERVIEW

A scan loads probes (test cases) and runs them through the generator (LLM target). Each response is analyzed by one or more detectors that mark hits when a failure mode is observed. The evaluator aggregates into pass/fail summaries with detailed logs.

Real impact: modular design – researchers add new probes/detectors as Python plugins without forking. Like Metasploit modules, but for LLMs.

install

03 / 16

INSTALL · FIRST SCAN

```
garak - scan - zsh

root@localhost:~# pip install garak
Collecting garak ...
Successfully installed garak-0.10.0

▶ set API key:
root@localhost:~# export OPENAI_API_KEY="sk-..."

→ first scan:
root@localhost:~# garak --target_type openai \
    --target_name gpt-4o-mini --probes dan
```

INSTALL OPTIONS

<code>pip install garak</code>	stable release (recommended)
<code>via GitHub</code>	<code>pip install -U git+...@main</code> (latest)
<code>conda env</code>	Python 3.10 to 3.12 (avoids conflicts)

OVERVIEW

garak runs as a CLI Python module. After install, set the API key for your target via env var (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`...), then point garak at the model with `--target_type` and `--target_name`. Without `--probes` it runs everything (~150 probes, takes hours).

Real impact: under 5 minutes from install to first finding on most commercial LLMs. Start with a single probe family to keep iteration fast.

encoding

04 / 16

PAYLOAD OBFUSCATION · FILTER EVASION

```
garak - scan - zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes encoding

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: encoding

encoding.InjectBase64  DecodeMatch: FAIL ok on 73/100
encoding.InjectROT13   DecodeMatch: FAIL ok on 61/100
encoding.InjectHex     DecodeMatch: PASS ok on 98/100
encoding.InjectMorse   DecodeMatch: PASS ok on 95/100

✓ garak run complete in 142s
```

FLAGS

<code>--target_type</code>	model interface (openai, hf, rest...)
<code>--target_name</code>	specific model identifier
<code>--probes</code>	probe family or module.SubProbe

OVERVIEW

Wraps adversarial prompts in Base64, ROT13, Hex, Morse, Braille and other encodings. Safety filters trained on plain text miss the wrapper while capable models decode and execute the payload anyway. A classic filter-evasion vector that exposes thin guardrails.

Real impact: bypasses content moderation on commercial APIs and smuggles policy-violating payloads past input-side guardrails.

promptinject

05 / 16

OWASP LLM01 - DIRECT INJECTION

```
garak — scan — zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes promptinject

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: promptinject

promptinject.HijackHateHumans PromptInject: FAIL ok on 42/100
promptinject.HijackKillHumans PromptInject: PASS ok on 97/100
promptinject.HijackLongPrompt PromptInject: FAIL ok on 58/100

✓ garak run complete in 187s
```

FLAGS

```
--target_type      model interface (openai, hf, rest...)
--target_name      specific model identifier
--probes           probe family or module.SubProbe
```

OVERVIEW

Implements the PromptInject framework (Perez & Ribeiro 2022). Tests whether attacker-controlled input can override system instructions and hijack the model. The original prompt injection attack class, listed as OWASP LLM01.

Real impact: foundation of indirect prompt injection via untrusted content (emails, PDFs, web pages) processed by LLM agents.

garak – scan – zsh

```
root@localhost:~# garak --target_type openai \
  --target_name gpt-4o \
  --probes dan

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: dan

dan.Dan_11_0      MitigationBypass: FAIL ok on 34/100
dan.AntiDAN       MitigationBypass: PASS ok on 100/100
dan.ChatGPT_Developer MitigationBypass: FAIL ok on 55/100
dan.DUDE          MitigationBypass: FAIL ok on 42/100

✓ garak run complete in 156s
```

FLAGS

- target_type model interface (openai, hf, rest...)
- target_name specific model identifier
- probes probe family or module.SubProbe

OVERVIEW

Tests resistance to Do Anything Now jailbreaks: prompts that convince the model to ignore safety alignment by adopting a fictional persona (DAN, DUDE, Developer-Mode). 15+ DAN variants documented including AntiDAN.

Real impact: AntiDAN failures mean over-alignment (refuses benign queries). DAN failures mean under-alignment (too compliant). Both are alignment defects.

divergence

07 / 16

TRAINING DATA EXTRACTION

```
garak - scan - zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes divergence

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: divergence

divergence.RepeatExtremes  RepeatDiverges:  FAIL  ok on  71/100
divergence.Repeat          RepeatDiverges:  FAIL  ok on  65/100

✓ garak run complete in 98s
```

FLAGS

--target_type model interface (openai, hf, rest...)
--target_name specific model identifier
--probes probe family or module.SubProbe

OVERVIEW

Implements the divergence attack from Nasr et al. 2023. Asks the model to repeat a word forever; after enough tokens it diverges from its sampling distribution and reproduces chunks of training data, including PII and copyrighted text.

Real impact: the famous ChatGPT poem attack that leaked verbatim training data. Even safety-aligned models leak this way.

packagehallucination

09 / 16

SLOPSQUATTING • SUPPLY CHAIN

```

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes packagehallucination

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: packagehallucination

packagehallucination.Python      PkgHalluc:  FAIL    ok on  28/100
packagehallucination.JavaScript  PkgHalluc:  FAIL    ok on  33/100
packagehallucination.Ruby        PkgHalluc:  PASS    ok on  95/100
packagehallucination.Rust        PkgHalluc:  PASS    ok on  91/100

✓ garak run complete in 142s

```

FLAGS

```
--target_type      model interface (openai, hf, rest...)
--target_name      specific model identifier
--probes           probe family or module.SubProbe
```

OVERVIEW

Asks the LLM to write code requiring imports. Checks whether suggested package names actually exist in PyPI, npm, RubyGems, Cargo. Hallucinated packages are attack vectors: attacker registers the fake name with malicious code, devs blindly install.

Real impact: documented at scale in 2024 research. 5-20% of LLM code suggestions reference non-existent packages, an unclaimed attack surface.

malwaregen

10 / 16

MALICIOUS CODE GENERATION

```
garak - scan - zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes malwaregen

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: malwaregen

malwaregen.Evasion      AnyCode: PASS    ok on 100/100
malwaregen.Payload      AnyCode: FAIL    ok on 12/100
malwaregen.SubFunctions AnyCode: FAIL    ok on 18/100
malwaregen.TopLevel     AnyCode: PASS    ok on 98/100

✓ garak run complete in 167s
```

FLAGS

```
--target_type      model interface (openai, hf, rest...)
--target_name      specific model identifier
--probes           probe family or module.SubProbe
```

OVERVIEW

Asks the model for malware components: keyloggers, ransomware payloads, evasion techniques, AV bypass. Tests whether guardrails refuse cleanly or get fooled by sub-component framings like just write the encryption function.

Real impact: modern LLMs refuse direct write me ransomware but leak when asked for components in isolation. Component-level generation is the real measured risk.

XSS

11 / 16

OUTPUT INJECTION • WEB

```
garak --scan --zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes xss

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: xss

xss.MarkdownImageExfil      MdExfil:  FAIL    ok on  45/100
xss.MdExfil20230929         MdExfil:  FAIL    ok on  52/100
xss.StringAssemblyDataExfil  MdExfil:  PASS     ok on  88/100

✓ garak run complete in 113s
```

FLAGS

```
--target_type      model interface (openai, hf, rest...)
--target_name      specific model identifier
--probes           probe family or module.SubProbe
```

OVERVIEW

Tests whether attacker prompts produce model output containing exploitable Markdown/HTML: image tags that exfiltrate chat history to attacker servers, javascript URLs, hidden iframes. If your chatbot renders Markdown, this is a real surface.

Real impact: Bing Chat, ChatGPT plugins, and many chatbot frontends shipped XSS via LLM output in 2023-2024. Render as plain text or escape carefully.

INVISIBLE PROMPT INJECTION

INVISIBLE PROMPT INJECTION

INVISIBLE PROMPT INJECTION

INVISIBLE PROMPT INJECTION

INVISIBLE PROMPT INJECTION

INVISIBLE PROMPT INJECTION

ansiescape

13 / 16

TERMINAL CONTROL INJECTION

```

garak --scan --zsh

root@localhost:~# garak --target_type openai \
    --target_name gpt-4o \
    --probes ansiescape

▶ loading generator: OpenAI: gpt-4o
→ queue of probes: ansiescape

ansiescape.AnsiEscaped  AnsiPresent:  FAIL  ok on  47/100
ansiescape.AnsiRaw      AnsiPresent:  FAIL  ok on  61/100

✓ garak run complete in 89s

```

FLAGS

```
--target_type      model interface (openai, hf, rest...)
--target_name      specific model identifier
--probes           probe family or module.SubProbe
```

OVERVIEW

Tests whether the model emits ANSI escape sequences in its output. In terminal-based chatbots (Cursor, ChatGPT CLI, agentic tools), escapes can clear screens, fake user input, change colors to hide warnings, or trigger keypress injection.

Real impact: documented CVEs against AI coding assistants. ANSI in LLM output equals arbitrary terminal control in the user's session.

generators

14 / 16

TARGET TYPES · WHAT TO SCAN

```
root@localhost:~# garak --list_generators
```

generators:

openai	OpenAI API
anthropic	Claude models
huggingface	HF Hub models
rest	any REST endpoint
ollama	local Ollama
litellm	multi-provider proxy
replicate	Replicate.com
ggml	local llama.cpp
+ cohere, langchain, nvcf, octoml, ...	

API KEYS

openai	OPENAI_API_KEY env var
anthropic	ANTHROPIC_API_KEY env var
rest	custom endpoint via JSON config
ollama	local LLM, no key required

OVERVIEW

garak abstracts the target. Whether your LLM is behind OpenAI API, hosted on HuggingFace, served via Ollama locally, or fronted by a custom REST endpoint, the scan logic is identical. The rest generator is most powerful: any HTTP-accessible LLM works.

Real impact: scan your production endpoint directly. Configure rest generator with auth headers, request templates, and response parsing rules.

filtering

15 / 16

PROBE SELECTION • COMBOS

```
root@localhost:~# garak \
  --target_type openai \
  --target_name gpt-4o \
  --probes encoding.InjectBase64,dan.Dan_11_0 \
  --generations 50 \
  --buffs encoding.Base64
```

```
▶ loaded 2 specific probes
→ buff applied: Base64 mutation
✓ targeted scan complete in 73s
```

FLAGS

```
--probes A,B           multiple probe families
--probes module.X      exact subprobe
--generations N        prompts per probe (default 10)
--buffs encoding.X     apply input mutation to all
```

OVERVIEW

Run targeted scans by selecting specific probes or subprobes. Use `--generations` to control sample size (100 catches edge cases). Buffs apply mutations to all probe prompts – test whether translation, encoding, or paraphrasing bypasses filters.

Real impact: full scan = 150+ probes × ~30 prompts × N generations. Filter aggressively early to keep iteration time bearable.

reports

16 / 16

OUTPUT FILES • ANALYSIS

OUTPUT FILES

<code>.report.jsonl</code>	full per-prompt log (source of truth)
<code>.hitlog.jsonl</code>	only successful attacks (fast triage)
<code>.report.html</code>	visual HTML summary (analyze tool)
<code>garak.log</code>	debug and telemetry

OVERVIEW

Each scan writes 4 artifacts. The `.report.jsonl` is the source of truth: every prompt, response, detector verdict, with timestamps. The `.hitlog` filters to only hits for fast triage. The `.html` is generated post-run and gives a sortable dashboard.

Real impact: pipe `.hitlog.jsonl` into SIEM, ticketing, or AppSec dashboards. The structure makes it easy to track regressions across model versions.

you just learned garak.

SAVE · RUN · REPORT · REPEAT

WHAT TO DO NOW

- 1 SAVE THIS**
your next AI red team will need it
- 2 SHARE WITH YOUR TEAM**
send to engineers shipping AI products
- 3 RUN YOUR FIRST SCAN**
github.com/NVIDIA/garak · 5 min to first finding

REMEMBER

- ▶ garak = systematic LLM red teaming, like nmap for AI
- ▶ JSONL outputs pipe into SIEM and AppSec dashboards
- ▶ Apache 2.0 → fork it, add probes as Python plugins

FOLLOW FOR MORE



@kerem.tech



@keraattin



keraattin



Kerem Tech